

PostgreSQL Views

What is a View?

- A **view** is a **virtual table** based on the result of an SQL query.
- It **does not store data physically**, only the query definition.
- When queried, it fetches the latest data from the underlying tables.

Create the book table

```
CREATE TABLE public.book (  
    bookid    SERIAL PRIMARY KEY,  
    title     VARCHAR(100),  
    author    VARCHAR(100),  
    book_cost NUMERIC(8,2)  
);
```

This creates a table named **book** with:

- bookid – auto-incrementing primary key
- title – book title
- author – author name
- book_cost – price (up to 8 digits, 2 decimals)

Insert sample data

```
INSERT INTO public.book (title, author, book_cost) VALUES  
( 'PostgreSQL Basics', 'Alice Johnson', 150.00),
```

```
('Advanced PostgreSQL', 'Bob Smith', 250.00),  
( 'SQL Performance Tuning', 'Carol White', 300.00),  
( 'Database Design', 'David Brown', 180.00);
```

Check the data

```
SELECT * FROM public.book;
```

Creating a View

Syntax:

```
CREATE [OR REPLACE] VIEW view_name AS  
  
SELECT column1, column2, ...  
  
FROM table_name  
  
WHERE condition;
```

Example:

```
CREATE VIEW expensive_books AS  
  
SELECT bookid, title, author, book_cost  
  
FROM public.book  
  
WHERE book_cost > 200;
```

Using a View

Query the view like a normal table:

```
SELECT * FROM expensive_books;
```

This always shows **up-to-date** data from the base table.

Changing a View

To modify the query:

```
CREATE OR REPLACE VIEW expensive_books AS
```

```
SELECT bookid, book_cost
```

```
FROM public."expensive_books"
```

```
WHERE book_cost > 250;
```

Updates the view definition without dropping it.

Renaming a View

```
ALTER VIEW expensive_books RENAME TO premium_books;
```

Dropping a View

```
DROP VIEW [IF EXISTS] premium_books;
```

IF EXISTS prevents an error if the view does not exist.

Key Points about Views

- Views **always reflect live data**.
- They **cannot be indexed** directly (indexes belong to the base tables).
- For heavy queries that rarely change, **Materialized Views** are better.

PostgreSQL Materialized Views

What is a Materialized View?

- A **materialized view** is like a **snapshot** of a query result.
- It **stores the data physically**, unlike a normal view.
- Faster to query for large datasets, but data is **not automatically updated**.

Creating a Materialized View

Syntax:

```
CREATE MATERIALIZED VIEW view_name AS  
  
SELECT columns  
  
FROM table_name  
  
WHERE condition;
```

Example:

```
CREATE MATERIALIZED VIEW expensive_books_mv AS  
  
SELECT bookid, title, author, book_cost  
  
FROM public.book  
  
WHERE book_cost > 200;
```

Stores the result set in the database.

Using a Materialized View

Query it like a table:

```
SELECT * FROM expensive_books_mv;
```

Shows **old data** until refreshed.

Refreshing a Materialized View

To update its data:

```
REFRESH MATERIALIZED VIEW expensive_books_mv;
```

✓ Re-runs the query and updates the snapshot.

- ♦ **Option:** To allow reading during refresh:

```
CREATE UNIQUE INDEX expensive_books_mv_idx ON  
expensive_books_mv(bookid);
```

```
REFRESH MATERIALIZED VIEW CONCURRENTLY expensive_books_mv;
```

(Requires a unique index on the materialized view)

Dropping a Materialized View

```
DROP MATERIALIZED VIEW IF EXISTS expensive_books_mv;
```

View vs Materialized View

Feature	View	Materialized View
Data storage	No (virtual table)	Yes (snapshot)
Data freshness	Always current	Needs manual refresh
Query performance	Slower for large queries	Faster after initial creation
Indexes on result set	Not possible	Possible

Practice Questions

1. Create a normal view to display books costing more than 300.
2. Create a materialized view to store the same result.
3. How do you refresh a materialized view to get updated data?
4. What is the difference between CREATE VIEW and CREATE MATERIALIZED VIEW?
5. Can you use indexes on a normal view? Why or why not?

Summary

- **View** → Virtual table, always shows latest data, no physical storage.
- **Materialized View** → Stores query results physically, faster to query, but must be **refreshed** to update.
- Use **normal views** for live data and **materialized views** for large or complex reports where performance is critical.

PostgreSQL Indexes

What are PostgreSQL Indexes?

- Indexes are **special tools** used to enhance data retrieval from databases.
- A database index is like the **index of a book** — it creates access paths for values in indexed columns.
- Helps PostgreSQL **locate rows faster** than scanning the whole table.
- Must be used properly to get performance benefits.

Features of PostgreSQL Indexes

- Improves data output with **SELECT** and **WHERE**.
- Can slow down **INSERT** and **UPDATE** (because indexes must also be updated).
- We can **CREATE** and **DROP** indexes without affecting data.
- **CREATE INDEX** command is used to define index name, table, and column.
- Supports **unique indexes**, similar to UNIQUE constraints.

Commands in PostgreSQL Indexes

Command	Description
Create Index	Create a new index on a table/column.
Drop Index	Delete an existing index.

List Indexes	Display all indexes in the database.
Unique Index	Create indexes with unique values.
Index on Expression	Create index based on an expression.
Partial Index	Index only a subset of rows (with condition).
Re-index	Rebuild one or more indexes.
Multicolumn Indexes	Create indexes on multiple columns for complex queries.

Types of PostgreSQL Indexes

1. Hash Indexes

- Works with **equality (=) operator** only.

Syntax:

```
CREATE INDEX index_name  
ON table_name  
USING HASH (column_name);
```

2. B-tree Indexes (Default & Most Common)

- Balanced tree structure; keeps data sorted.

- Allows **search, insert, delete** in logarithmic time.
- Supports operators: <, <=, =, >=, BETWEEN, IN, IS NULL, IS NOT NULL.
- Can also be used for **LIKE**, ~, ILIKE, and ~* when patterns start with anchors.
- Default type when no index type is specified.

3. GIN Indexes (Generalized Inverted Index)

- Useful when a column contains **multiple values** (arrays, jsonb, hstore, ranges).
- Ideal for **full-text search** and complex data types.

4. GiST Indexes (Generalized Search Tree)

- Allows custom data structures (geometric, full-text search, ranges).
- Flexible indexing for advanced queries.

5. SP-GiST (Space-Partitioned GiST)

- Supports **partitioned search trees** (non-balanced structures).
- Best for data with natural clustering: GIS, multimedia, phone routing, IP routing.

6. BRIN Indexes (Block Range Indexes)

- Lightweight, smaller, less costly than B-tree.
- Best for **large tables** with sequential/linear data (e.g., dates, timestamps).
- Maintains summary of data blocks instead of individual rows.

Disadvantages of PostgreSQL Indexes

- Should **not** be used on columns with many **NULL values**.
- Not useful for **small tables** (scanning is faster).
- Avoid indexes on columns updated very often.
- Not recommended for tables with frequent **bulk inserts/updates**.

Overview

- **Indexes speed up retrieval** but can slow down data modification.
- PostgreSQL supports many commands for managing indexes.
- Types include **Hash, B-tree, GIN, GiST, SP-GiST, BRIN**.
- Choosing the right index type is critical for performance.

PostgreSQL Create Index

Example: PostgreSQL Create Index

Suppose we have a table:

```
CREATE TABLE Students (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100),  
    email VARCHAR(150),  
    course VARCHAR(50)  
);
```

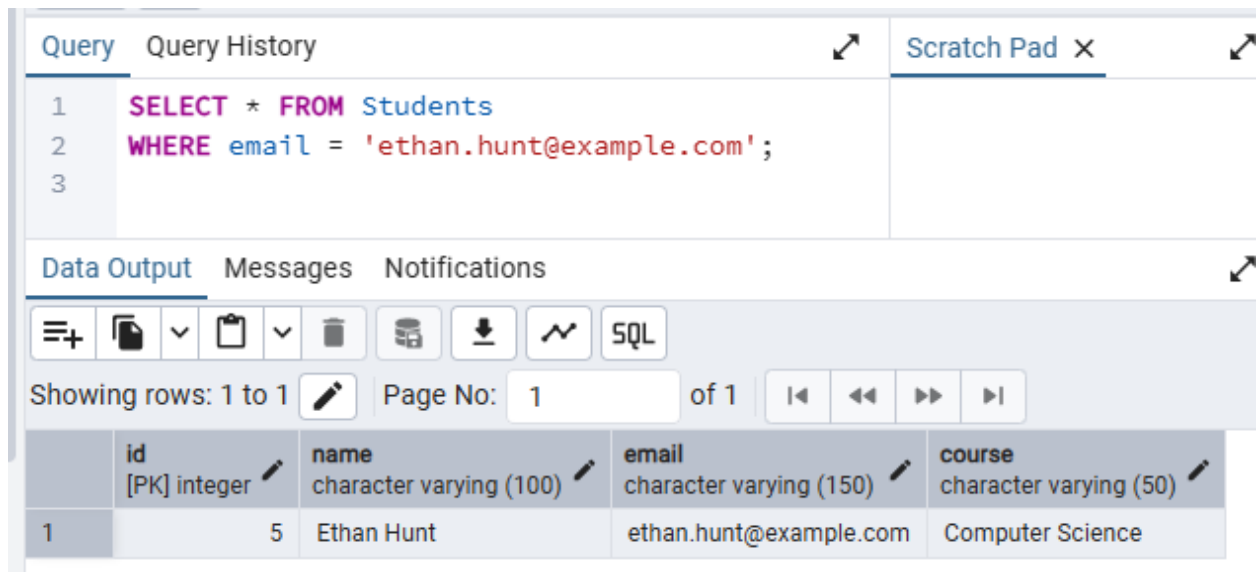
```
INSERT INTO Students (name, email, course) VALUES  
(  
    'Alice Johnson', 'alice.johnson@example.com', 'Computer Science'),  
(  
    'Bob Smith', 'bob.smith@example.com', 'Mathematics'),  
(  
    'Charlie Brown', 'charlie.brown@example.com', 'Computer Science'),  
(  
    'Diana Prince', 'diana.prince@example.com', 'Physics'),  
(  
    'Ethan Hunt', 'ethan.hunt@example.com', 'Computer Science'),  
(  
    'Fiona White', 'fiona.white@example.com', 'Mathematics'),  
(  
    'George Miller', 'george.miller@example.com', 'Computer Science'),  
(  
    'Hannah Scott', 'hannah.scott@example.com', 'Physics'),  
(  
    'Ivan Davis', 'ivan.davis@example.com', 'Computer Science'),  
(  
    'Julia Roberts', 'julia.roberts@example.com', 'Mathematics');
```

Without Index (Sequential Scan)

We want to find the student with a specific email:

```
SELECT * FROM Students
```

```
WHERE email = 'ethan.hunt@example.com';
```



The screenshot shows a PostgreSQL query editor interface. The 'Query' tab is active, displaying the following SQL query:

```
1 SELECT * FROM Students
2 WHERE email = 'ethan.hunt@example.com';
3
```

Below the query editor, the 'Data Output' tab is active, showing the results of the query. The results are displayed in a table with the following columns: id, name, email, and course. The table shows one row of data for the student Ethan Hunt.

	id [PK] integer	name character varying (100)	email character varying (150)	course character varying (50)
1	5	Ethan Hunt	ethan.hunt@example.com	Computer Science

- PostgreSQL scans the **entire Students table** row by row.
- We can confirm with:

```
EXPLAIN SELECT *
```

```
FROM Students
```

```
WHERE email = 'ethan.hunt@example.com';
```

Query
Query History
Scratch Pad X

1 EXPLAIN SELECT *
2 FROM Students
3 WHERE email = 'ethan.hunt@example.com';

Data Output
Messages
Notifications

Showing rows: 1 to 2 of 1
Page No: 1
SQL

QUERY PLAN
1 Seq Scan on students (cost=0.00..11.38 rows=1 width=658)
2 Filter: ((email)::text = 'ethan.hunt@example.com'::text)

Creating an Index

Now, let's create an index on the email column:

```
CREATE INDEX idx_students_email
```

```
ON Students (email);
```

Query
Query History
Scratch Pad X

1 CREATE INDEX idx_students_email
2 ON Students (email);
3

Data Output
Messages
Notifications

CREATE INDEX

Query returned successfully in 94 msec.

- PostgreSQL creates idx_students_email.
- This makes lookups on email much faster.

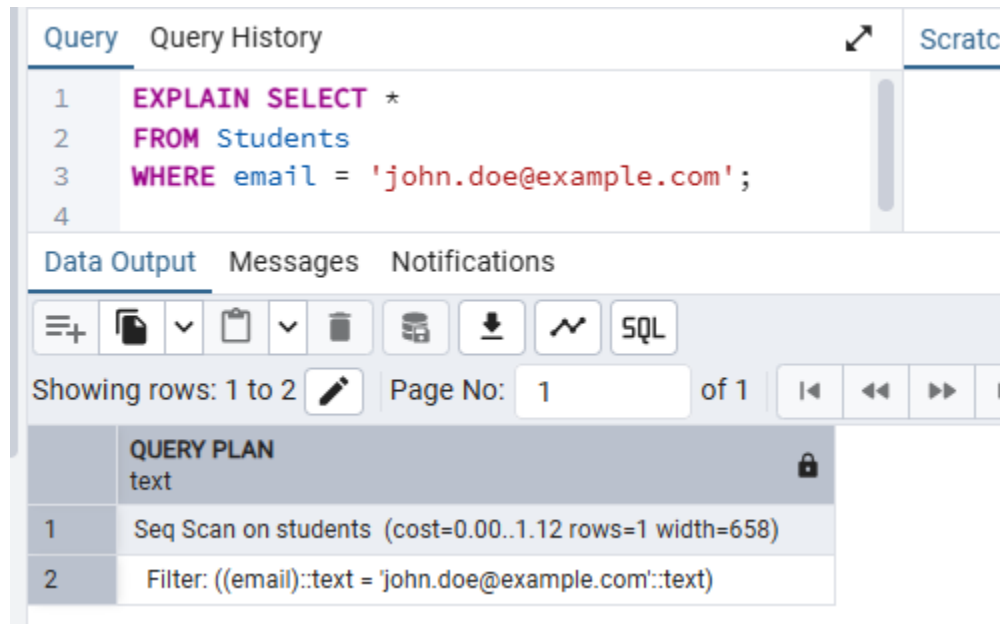
With Index (Index Scan)

Run the same query again:

```
EXPLAIN SELECT *
```

```
FROM Students
```

```
WHERE email = 'john.doe@example.com';
```



The screenshot shows a database query interface. The top section is titled 'Query' and contains the following SQL query:

```
1 EXPLAIN SELECT *
2 FROM Students
3 WHERE email = 'john.doe@example.com';
4
```

Below the query is a section titled 'Data Output' with tabs for 'Messages' and 'Notifications'. The 'Data Output' tab is active, showing a table with the following data:

	QUERY PLAN
1	Seq Scan on students (cost=0.00..1.12 rows=1 width=658)
2	Filter: ((email)::text = 'john.doe@example.com'::text)

Output: **Index Scan using idx_students_email** (much faster).

Another Example (Multi-column Index)

Suppose we often search by both **course** and **name**:

```
CREATE INDEX idx_students_course_name
```

```
ON Students (course, name);
```

Query
Query History
Scratch Pad X

1 CREATE INDEX idx_students_course_name
2 ON Students (course, name);
3

Data Output
Messages
Notifications

CREATE INDEX

Query returned successfully in 96 msec.

Now queries like:

SELECT * FROM Students

WHERE course = 'Computer Science' AND name = 'Alice Johnson';

Query
Query History

1 SELECT * FROM Students
2 WHERE course = 'Computer Science' AND name = 'Alice Johnson';
3

Data Output
Messages
Notifications

Showing rows: 1 to 1
Page No

	id [PK] integer	name character varying (100)	email character varying (150)	course character varying (50)
1	1	Alice Johnson	alice.johnson@example.com	Computer Science

Will use idx_students_course_name instead of scanning the entire table.

Overview

- Example changed to Students table with id, name, email, and course.
- EXPLAIN helps check whether PostgreSQL is using **sequential scan** or **index scan**.
- Indexes speed up lookups, especially for large tables.

PostgreSQL Drop Index

Why use DROP INDEX?

- The DROP INDEX command is used to **remove an existing index** from a table.
- Removing unnecessary indexes can **improve performance**, since indexes consume space and add overhead during INSERT, UPDATE, and DELETE operations.

Syntax of DROP INDEX

DROP INDEX [CONCURRENTLY]

[IF EXISTS] index_name

[CASCADE | RESTRICT];

Parameters

Parameter	Description
index_name	The name of the index to be deleted.
IF EXISTS	Prevents error if the index does not exist.
CASCADE	Automatically removes dependent objects.
RESTRICT	Default option; prevents dropping if dependent objects exist.
CONCURRENTLY	Drops the index without locking the table for writes.


```
DROP INDEX idx1, idx2, idx3;
```

Step 1: View Data in Students Table

Query Query History ↗

```
1 SELECT * FROM Students;
2
```

Data Output Messages Notifications

≡+ 📄 ▼ 📋 ▼ 🗑️ 💾 ⬇️ 📈 SQL
Showing rows: 1 to 10 ✎
Page

	id [PK] integer ✎	name character varying (100) ✎	email character varying (150) ✎	course character varying (50) ✎
1	1	Alice Johnson	alice.johnson@example.c...	Computer Science
2	2	Bob Smith	bob.smith@example.com	Mathematics
3	3	Charlie Brown	charlie.brown@example.c...	Computer Science
4	4	Diana Prince	diana.prince@example.co...	Physics
5	5	Ethan Hunt	ethan.hunt@example.com	Computer Science
6	6	Fiona White	fiona.white@example.com	Mathematics
7	7	George Miller	george.miller@example.c...	Computer Science
8	8	Hannah Scott	hannah.scott@example.c...	Physics
9	9	Ivan Davis	ivan.davis@example.com	Computer Science
10	10	Julia Roberts	julia.roberts@example.com	Mathematics

```
CREATE INDEX idx_students_course
```

Index created successfully on course.

Step 3: Run a Query

```
EXPLAIN SELECT *
```

```
FROM Students
```

```
WHERE course = 'Physics';
```

The screenshot shows a database query interface with a 'Query' tab selected. The query text is: `EXPLAIN SELECT * FROM Students WHERE course = 'Physics';`. Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, displaying a 'QUERY PLAN' table. The table has two rows: 1. 'Seq Scan on students (cost=0.00..1.12 rows=1 width=65...)' and 2. 'Filter: ((course)::text = 'Physics'::text)'. The interface also includes a toolbar with icons for query execution, saving, and other database functions.

	QUERY PLAN
1	Seq Scan on students (cost=0.00..1.12 rows=1 width=65...)
2	Filter: ((course)::text = 'Physics'::text)

- PostgreSQL **may or may not use the index** depending on query planner decisions.
- If the table is small, a **sequential scan** may be chosen.

Step 4: Drop the Index

```
DROP INDEX idx_students_course;
```

The screenshot shows a database query interface with a 'Query' tab selected. The query text is: `DROP INDEX idx_students_course;`. Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, displaying the message: 'DROP INDEX' and 'Query returned successfully in 93 msec.'.

```
DROP INDEX
```

Query returned successfully in 93 msec.

Step 5: Drop Index Safely (Optional)

DROP INDEX IF EXISTS idx_students_course;

Query	Query History
1	DROP INDEX IF EXISTS idx_students_course;
2	
Data Output	Messages
NOTICE: index "idx_students_course" does not exist, skipping DROP INDEX	
Query returned successfully in 68 msec.	

- Avoids errors if index does not exist.

Overview

- **DROP INDEX** removes existing indexes.
- Use **IF EXISTS** to avoid errors.
- **CASCADE** drops dependent objects; **RESTRICT** (default) prevents accidental deletion.
- **CONCURRENTLY** drops without locking the table but cannot be used inside transactions.

practice questions for PostgreSQL **DROP INDEX**:

1. Create a table **Books** with columns (**book_id**, **title**, **author**, **published_year**). Add an index on the author column. Then drop that index.
2. Create a unique index on the **email** column of a **Users** table. Later, drop the index using **IF EXISTS**.
3. On the **Orders** table, create a composite index on (**customer_id**, **order_date**). Then drop it.
4. Create two indexes on the **Employees** table (**emp_name**, **department**). Drop both indexes in a single **DROP INDEX** command.
5. Create an index on the **city** column of a **Customers** table. Run a query using **EXPLAIN** to check if the index is being used, then drop the index.

PostgreSQL List Indexes

What are Indexes?

- An **index** is a database object that improves the **speed of data retrieval**.
- It works like a **book's index** – instead of scanning every page, PostgreSQL can jump directly to the right location.
- But remember: indexes also **add overhead** during INSERT, UPDATE, and DELETE operations because PostgreSQL must maintain them.

Why List Indexes?

- To **check existing indexes** before creating new ones (avoid duplicates).
- To **analyze performance** (see if an index exists for queries).
- To **identify unused or redundant indexes** that can be dropped to improve performance.

Ways to List Indexes in PostgreSQL

1. Using psql command

- Connect to database:

```
\c akshay;
```

- Show table structure with indexes:

```
\d employee;
```

```

postgres=# \c akshay;
You are now connected to database "akshay" as user "postgres".
akshay=# \d employee;
               Table "public.employee"
  Column |          Type          | Collation | Nullable | Default
-----+-----+-----+-----+-----
 id      | integer                |           | not null |
 name    | character varying(50)  |           |          |
 email   | character varying(100) |           |          |
Indexes:
    "employee_pkey" PRIMARY KEY, btree (id)

```

Displays columns, constraints, triggers, and indexes.

2. Using pg_indexes View

pg_indexes is a system catalog view storing all index information.

Column	Description
schemaname	Schema name of table & index
tablename	Table name
indexname	Index name
tablespace	Tablespace (if used)
indexdef	Full CREATE INDEX definition

(a) All indexes in schema public

```
SELECT tablename, indexname, indexdef
```

```
FROM pg_indexes
```

WHERE schemaname = 'public'

ORDER BY tablename, indexname;

The screenshot shows a SQL query editor with a query window and a data output window. The query window contains the following SQL code:

```
1 SELECT tablename, indexname, indexdef
2 FROM pg_indexes
3 WHERE schemaname = 'public'
4 ORDER BY tablename, indexname;
5
```

The data output window shows the results of the query, displaying three rows of index information for the 'public' schema:

	tablename name	indexname name	indexdef text
1	students	idx_students_course_name	CREATE INDEX idx_students_course_name ON public.students USING btree (course, name)
2	students	idx_students_email	CREATE INDEX idx_students_email ON public.students USING btree (email)
3	students	students_pkey	CREATE UNIQUE INDEX students_pkey ON public.students USING btree (id)

(b) Indexes for a specific table

SELECT indexname, indexdef

FROM pg_indexes

WHERE tablename = 'employee';

The screenshot shows a SQL query editor with a query window and a data output window. The query window contains the following SQL code:

```
1 SELECT indexname, indexdef
2 FROM pg_indexes
3 WHERE tablename = 'employee';
4
```

The data output window shows the results of the query, displaying one row of index information for the 'employee' table:

	indexname name	indexdef text
1	employee_pkey	CREATE UNIQUE INDEX employee_pkey ON public.employee USING btree (employee_id)

Let's add some sample data and indexes to your employee table.

1. Create the table

```
CREATE TABLE employee (  
    emp_id SERIAL PRIMARY KEY,  
    emp_name VARCHAR(50),  
    phone VARCHAR(15),  
    address VARCHAR(100)  
);
```

2. Insert sample rows

```
INSERT INTO employee (emp_name, phone, address) VALUES
('Alice Johnson', '555-1111', 'New York'),
('Bob Smith', '555-2222', 'Boston'),
('Charlie Brown', '555-3333', 'Chicago'),
('David Lee', '555-4444', 'Boston'),
('Emma Davis', '555-5555', 'New York');
```

3. Create some indexes

-- Index on address column

```
CREATE INDEX idx_employee_address ON employee(address);
```

-- Index on phone column

```
CREATE INDEX idx_employee_phone ON employee(phone);
```

```
SELECT indexname, indexdef
```

```
FROM pg_indexes
```

```
WHERE tablename = 'employee';
```

(c) Indexes for tables starting with e

```
SELECT tablename, indexname, indexdef
```

```
FROM pg_indexes
```

```
WHERE tablename LIKE 'e%'
```

```
ORDER BY tablename, indexname;
```

Key Points to Remember

- PostgreSQL does **not** support SHOW INDEXES like MySQL.
- Use \d table_name in **psql** for a quick look.
- Use pg_indexes for **detailed index information**.
- Regularly check indexes to ensure they are **actually used** (EXPLAIN helps with this).

PostgreSQL UNIQUE Index

What is a UNIQUE Index?

- A **UNIQUE index** ensures that the values in one or more columns are **unique**.
- It prevents duplicate values and helps maintain **data integrity**.
- PostgreSQL automatically creates a unique index when you define:
 - **PRIMARY KEY**
 - **UNIQUE constraint**
- **NULL values** are treated as different, so multiple NULLs are allowed.

Syntax

```
CREATE UNIQUE INDEX index_name  
ON table_name (column_name);
```

Example – Creating Table with Unique Index

```
DROP TABLE IF EXISTS Consumer;
```

```
CREATE TABLE Consumer (  
    consumer_id SERIAL PRIMARY KEY,  
    first_name VARCHAR(255) NOT NULL,  
    last_name VARCHAR(255) NOT NULL,  
    mobile_number VARCHAR(20) UNIQUE  
);
```


Here:

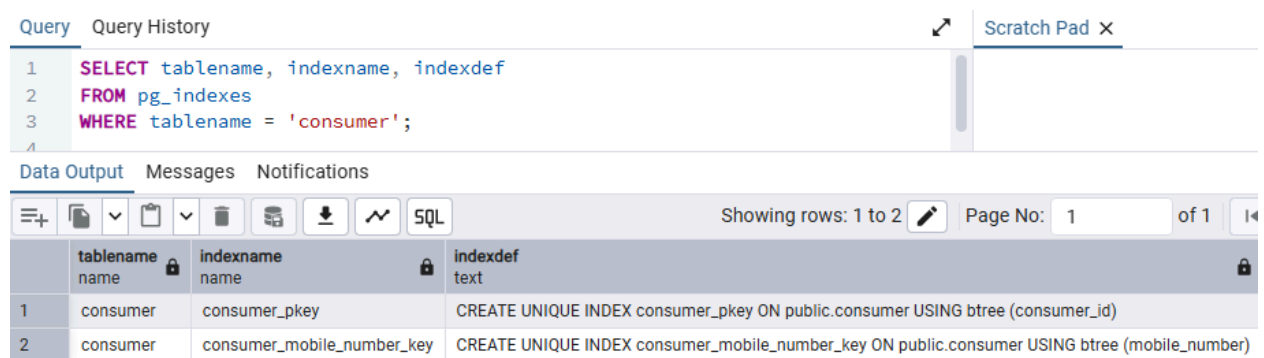
- consumer_id → PRIMARY KEY (unique index created automatically).
- mobile_number → UNIQUE (unique index created automatically).

View Indexes of a Table

```
SELECT tablename, indexname, indexdef
```

```
FROM pg_indexes
```

```
WHERE tablename = 'consumer';
```



The screenshot shows a database query tool interface. The query editor contains the following SQL code:

```
1 SELECT tablename, indexname, indexdef
2 FROM pg_indexes
3 WHERE tablename = 'consumer';
```

The results are displayed in a table with the following columns: tablename, indexname, and indexdef. The table shows two indexes for the 'consumer' table.

	tablename name	indexname name	indexdef text
1	consumer	consumer_pkey	CREATE UNIQUE INDEX consumer_pkey ON public.consumer USING btree (consumer_id)
2	consumer	consumer_mobile_number_key	CREATE UNIQUE INDEX consumer_mobile_number_key ON public.consumer USING btree (mobile_number)

Single-Column UNIQUE Index

Step 1 – Add a new column

```
ALTER TABLE Consumer
```

```
ADD email VARCHAR(255);
```

Step 2 – Create UNIQUE index

```
CREATE UNIQUE INDEX idx_consumer_email
```

```
ON Consumer(email);
```

Step 3 – Insert data

Valid insert:

```
INSERT INTO Consumer(first_name, last_name, mobile_number, email)
VALUES ('Mike','Ross','(305)-444-5432','mike@javatpoint.com');
```

Invalid insert (duplicate email):

```
INSERT INTO Consumer(first_name, last_name, mobile_number, email)
VALUES ('Mike','Taylor','(305)-333-9876','mike@javatpoint.com');
```

Error: duplicate key value violates unique constraint "idex_consumer_email"

Multi-Column UNIQUE Index

Step 1 – Add columns

```
ALTER TABLE Consumer
ADD address VARCHAR(255),
ADD work_address VARCHAR(200);
```

Step 2 – Create multi-column unique index

```
CREATE UNIQUE INDEX idex_consumer_address
ON Consumer(address, work_address);
```

Step 3 – Insert data

Valid insert:

```
INSERT INTO Consumer(first_name, last_name, address, work_address)
VALUES ('Olivia','Smith','Boston','Brookline');
```

Different address + same work_address (allowed):

```
INSERT INTO Consumer(first_name, last_name, address, work_address)
VALUES ('Olivia','Smith','Los Angeles','Brookline');
```

Same address + work_address (not allowed):

```
INSERT INTO Consumer(first_name, last_name, address, work_address)
```

```
VALUES ('William','Smith','Los Angeles','Brookline');
```

Error: duplicate key value violates unique constraint "index_consumer_address"

Key Points (Overview)

- Ensures **uniqueness** of column(s).
- PostgreSQL **creates unique index automatically** for PRIMARY KEY and UNIQUE constraints.
- **Single-column** unique index → restricts duplicates in one column.
- **Multi-column** unique index → restricts duplicates in **combination of columns**.
- **Multiple NULLs allowed** since each is treated as different.

PostgreSQL Index on Expression

What is an Index on Expression?

- Normally, indexes are created on **columns**.
- But in PostgreSQL, we can create an index on an **expression** (computed value).
- These are also called **functional indexes**.
- They improve query performance when conditions involve expressions in WHERE or ORDER BY.

Note:

- Expression indexes require PostgreSQL to evaluate the expression for each row → **maintenance cost is higher**.
- Use them when **query speed is more important** than insert/update speed.

Syntax

```
CREATE INDEX index_name  
ON table_name (expression);
```

Parameters:

- **index_name** → name of the index.
- **table_name** → table where index is created.
- **expression** → expression involving column(s).

Example – Case-Insensitive Search

Suppose we have an employee table:

```
SELECT * FROM employee;
```

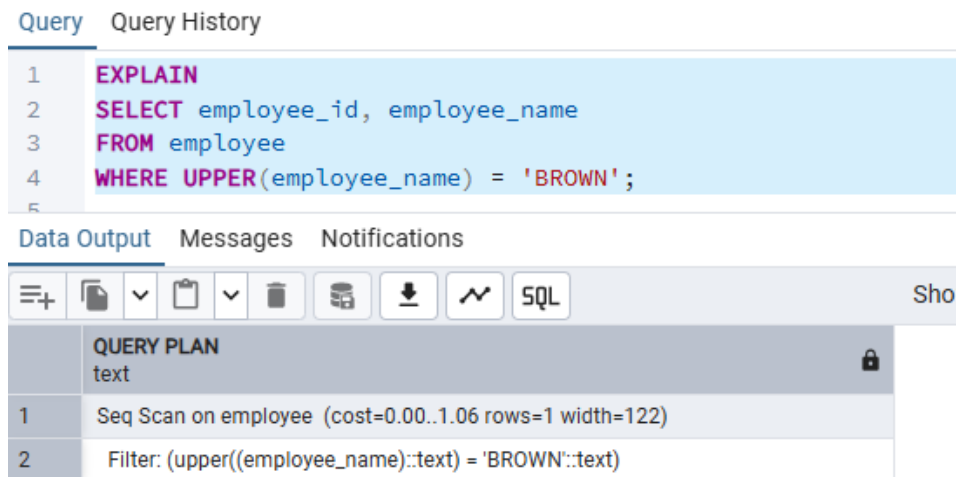
Without Expression Index

```
EXPLAIN
```

```
SELECT employee_id, employee_name
```

```
FROM employee
```

```
WHERE UPPER(employee_name) = 'BROWN';
```



The screenshot shows a database query tool interface. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying the following SQL query:

```
1 EXPLAIN
2 SELECT employee_id, employee_name
3 FROM employee
4 WHERE UPPER(employee_name) = 'BROWN';
5
```

Below the query, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing a table with the following content:

	QUERY PLAN
1	Seq Scan on employee (cost=0.00..1.06 rows=1 width=122)
2	Filter: (upper((employee_name)::text) = 'BROWN'::text)

At the bottom right of the 'Data Output' tab, there is a 'Show' button.

Create Index on Expression

```
CREATE INDEX idx_ic_emp_lname
```

```
ON employee (UPPER(employee_name));
```

```
Query  Query History
1  CREATE INDEX idx_ic_emp_lname
2  ON employee (UPPER(employee_name));
3

Data Output  Messages  Notifications

CREATE INDEX

Query returned successfully in 90 msec.
```

With Expression Index

EXPLAIN

SELECT employee_id, employee_name

FROM employee

WHERE UPPER(employee_name) = 'BROWN';

```
Query  Query History
1  EXPLAIN
2  SELECT employee_id, employee_name
3  FROM employee
4  WHERE UPPER(employee_name) = 'BROWN';
5

Data Output  Messages  Notifications

QUERY PLAN
text
1  Seq Scan on employee (cost=0.00..1.06 rows=1 width=122)
2  Filter: (upper((employee_name)::text) = 'BROWN'::text)
```

Key Points

Index on Expression = index created on function or computed value of column(s).

- Useful for queries with functions like UPPER(), LOWER(), calculations, or expressions.
- Syntax: CREATE INDEX index_name ON table(expression);
- **Tradeoff**: Faster queries but **slower inserts/updates**.

PostgreSQL Partial Index

What is a Partial Index?

- A **Partial Index** is an index built on a **subset of rows** in a table.
- Defined using a **WHERE condition** → only rows matching that condition are indexed.
- Helps in:
 - Reducing index size
 - Speeding up queries (especially when filtering by constant values)

Useful when queries often filter on specific conditions like:

```
SELECT *
```

```
FROM table_name
```

```
WHERE column_name = constant_value;
```

Syntax

```
CREATE INDEX index_name
```

```
ON table_name(column_list)
```

```
WHERE condition;
```

- **index_name** → name of the index.
- **column_list** → column(s) to index.
- **condition** → predicate that defines which rows to include.

Example – Buyer Table

Step 1 – Create Table

```
CREATE TABLE Buyer (  
    Buyer_id INT PRIMARY KEY,  
    first_name VARCHAR(35) NOT NULL,  
    last_name VARCHAR(35) NOT NULL,  
    email_address VARCHAR(50),  
    active INT  
);
```

Step 2 – Insert Data

```
INSERT INTO Buyer (Buyer_id, first_name, last_name, email_address, active)  
VALUES  
(1001,'Mike', 'Ross','mike.ross11@hotmail.com',1),  
(1002,'Olivia', 'Smith','O.smith@gmail.com',0),  
(1003,'Kathy', 'James','kathy.james@gmail.com',0),  
(1004,'Annie', 'Russell','annie33@gmail.com',0),  
(1005,'Rachel', 'Barnes','barnes.rachel55@gmail.com',1);
```

Query Without Partial Index

Find all inactive buyers:

```
SELECT Buyer_id, first_name, last_name, email_address  
FROM Buyer  
WHERE active = 0;
```


Query

Query History

1

2

3

4

SELECT Buyer_id, first_name, last_name, email_address

FROM Buyer

WHERE active = 0;

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑️

🗄️

⬇️

📈

SQL

Showing rows: 1 to 3

✎

Page N

buyer_id
[PK] integer

first_name
character varying (35)

last_name
character varying (35)

email_address
character varying (50)

1

1002

Olivia

Smith

O.smith@gmail.com

2

1003

Kathy

James

kathy.james@gmail.com

3

1004

Annie

Russell

annie33@gmail.com

Using EXPLAIN shows a **full table scan** → inefficient for large tables.

Normal Index on Column

`CREATE INDEX idx_buyer_active`

`ON Buyer(active);`

Query	Query History
1	<code>CREATE INDEX idx_buyer_active</code>
2	<code>ON Buyer(active);</code>
3	

Data Output	Messages	Notifications
	CREATE INDEX	
	Query returned successfully in 103 msec.	

Improves queries but still indexes **all rows**, including those never searched (active buyers).

Partial Index (Better)

`CREATE INDEX idx_buyer_inactive`

ON Buyer(active)

WHERE active = 0;

```
1 CREATE INDEX idx_buyer_inactive
2 ON Buyer(active)
3 WHERE active = 0;
4
```

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 70 msec.

- ✓ Index only includes **inactive buyers**.
- ✓ Query planner now uses `idx_buyer_inactive` when filtering with `WHERE active = 0`.

Key Points (Overview)

- A **Partial Index** = index on a **subset of rows** defined by a condition.
- Reduces index size and increases performance.
- Best when queries often filter on **constant conditions**.
- Only used if the query's WHERE clause matches the **index condition**.

PostgreSQL REINDEX

What is REINDEX?

- PostgreSQL indexes can become **invalid or corrupted** due to:
 - Software bugs
 - Hardware failures
 - Data inconsistencies
- **REINDEX** is used to **rebuild one or more indexes** to restore data accuracy and performance.

It works like **DROP + CREATE INDEX**, but with different locking rules.

Syntax

```
REINDEX [ ( VERBOSE ) ] { INDEX | TABLE | SCHEMA | DATABASE | SYSTEM }  
name;
```

Parameters:

- **REINDEX** → keyword to rebuild indexes.
- **VERBOSE** → optional; shows progress messages.
- **INDEX** → rebuilds a single index.
- **TABLE** → rebuilds all indexes of a table.
- **SCHEMA** → rebuilds all indexes in a schema.
- **DATABASE** → rebuilds all indexes in a database.
- **SYSTEM** → rebuilds all indexes on system catalogs.

Examples

Reindex a Single Index

-- Create a sample table

```
CREATE TABLE products (  
    product_id SERIAL PRIMARY KEY,  
    product_name VARCHAR(100),  
    price NUMERIC  
);
```

-- Create an index on product_name

```
CREATE INDEX idx_products_name ON products(product_name);
```

-- Rebuild that specific index

```
REINDEX INDEX idx_products_name;
```

1. Rebuild a single index

```
REINDEX INDEX index_name;
```

Example

```
REINDEX INDEX idx_products_name;
```

2. Rebuild all indexes of a table

```
REINDEX TABLE table_name;
```

Example

```
REINDEX TABLE products;
```

3. Rebuild all indexes in a schema

```
REINDEX SCHEMA schema_name;
```

Example

```
REINDEX SCHEMA public;
```

4. Rebuild all indexes in a database

```
REINDEX DATABASE database_name;
```

Example

```
REINDEX DATABASE mybd;
```

5. Rebuild system catalog indexes

```
REINDEX SYSTEM database_name;
```

Example

```
REINDEX SYSTEM mydb;
```

REINDEX vs DROP INDEX + CREATE INDEX

Command	Behavior
REINDEX	Locks writes , but allows reads on the table while rebuilding.
DROP INDEX + CREATE INDEX	- DROP INDEX → locks both reads and writes (exclusive lock). - CREATE INDEX → locks writes , allows reads (some may be exclusive).

Key Points

- **REINDEX** rebuilds indexes when they become invalid or corrupted.
- Can be applied to a **single index, table, schema, database, or system catalogs**.
- Works similarly to **DROP + CREATE INDEX**, but uses a different locking mechanism (less blocking).
- Useful for **performance recovery** and **index maintenance**.

PostgreSQL Multi-column Indexes

What are Multi-column Indexes?

- An **index created on multiple columns** of a table.
- Also called:
 - Composite Index
 - Concatenated Index
 - Combined Index
- Supports up to **32 columns** (can be changed in pg_config_manual.h).
- Supported index types: **B-tree, GiST, GIN, BRIN**.

Use them when queries **filter or sort by multiple columns together**.

Syntax

```
CREATE INDEX index_name  
ON table_name (col1, col2, col3, ...);
```

Best Practice:

- Put the **most frequently used columns** in WHERE at the **start of the column list**.
- Order of columns **matters**.

When Optimizer Uses Multi-column Index

Works with:

```
WHERE col1 = value1 AND col2 = value2 AND col3 = value3;
```

```
WHERE col1 = value1 AND col2 = value2;
```

WHERE col1 = value1;

Not used with:

WHERE col3 = value3;

WHERE col2 = value2 AND col3 = value3;

Example

1. Create a Table

```
CREATE TABLE Person (  
    person_id INT GENERATED BY DEFAULT AS IDENTITY,  
    first_name VARCHAR(30) NOT NULL,  
    last_name VARCHAR(30) NOT NULL  
);
```

2. Insert Data

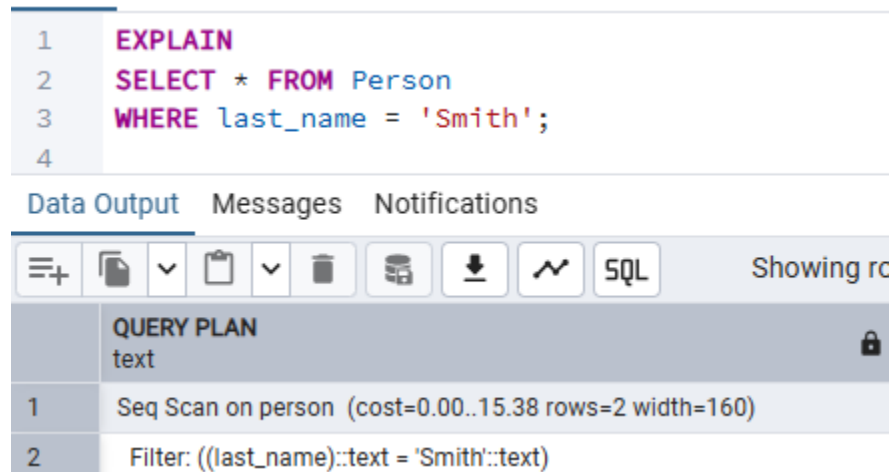
```
INSERT INTO Person (first_name, last_name)  
VALUES ('Mike', 'Ross'),  
    ('John', 'Smith'),  
    ('Owen', 'Norman'),  
    ('Bruce', 'Willis'),  
    ('Douglas', 'Smith'),  
    ('Olivia', 'Smith');
```

3. Query Without Index

EXPLAIN

SELECT * FROM Person

WHERE last_name = 'Smith';



The screenshot shows a database query execution interface. At the top, the SQL query is displayed: `1 EXPLAIN`, `2 SELECT * FROM Person`, `3 WHERE last_name = 'Smith';`, and `4`. Below the query, there are tabs for "Data Output", "Messages", and "Notifications". The "Data Output" tab is selected, showing a table with two columns: "QUERY PLAN" and "text". The table contains two rows:
1. "Seq Scan on person (cost=0.00..15.38 rows=2 width=160)"
2. "Filter: ((last_name)::text = 'Smith'::text)"

	QUERY PLAN	text
1	Seq Scan on person	(cost=0.00..15.38 rows=2 width=160)
2	Filter:	((last_name)::text = 'Smith'::text)

Multi-column B-tree Index

-- Create index on (last_name, first_name)

CREATE INDEX idx_person_names

ON Person (last_name, first_name);

Use Cases

-- Uses index (because last_name is first in order)

SELECT * FROM Person

WHERE last_name = 'Smith';

Query

Query History

1

SELECT * **FROM** Person

2

WHERE last_name = 'Smith';

3

Data Output

Messages

Notifications

≡

+

▼

▼

SQL

Showing

	person_id integer	first_name character varying (30)	last_name character varying (30)
1	2	John	Smith
2	5	Douglas	Smith
3	6	Olivia	Smith

-- Uses index (both columns in WHERE)

SELECT * FROM Person

WHERE last_name = 'Smith' AND first_name = 'John';

Query

Query History

1

2

3

SELECT * FROM Person

WHERE last_name = 'Smith' AND first_name = 'John';

Data Output

Messages

Notifications

≡

+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

Showing rows: 1 to

person_id

integer

first_name

character varying (30)

last_name

character varying (30)

1

2

John

Smith

-- Does NOT use index (first_name alone not leading column)

SELECT * FROM Person

WHERE first_name = 'John';

Trick:

SET enable_seqscan = off;

→ Forces PostgreSQL to use the index for testing.

Multi-column Index Types

B-tree

- Works **best with equality and range queries**.
- Column order **matters a lot**.

GIN

- Can be used with **any subset of columns**.
- Column order **does not affect efficiency**.

GiST

- Works with subsets of columns, but
- **First column should be selective** (many distinct values).
- Otherwise, index is inefficient.

BRIN

- Works like GIN → **subsets of columns supported**.
- Often better to use **multiple BRIN indexes** (with different `pages_per_range`) than one big multi-column BRIN index.

Key Notes

- **Choose columns wisely** (based on query patterns).
- In many cases, **single-column indexes** are more efficient.
- Use multi-column indexes when queries often filter by **multiple columns together**.

Summary:

- Multi-column indexes = composite indexes.

- Only supported types: **B-tree, GiST, GIN, BRIN**.
- Column **order is critical** (except for GIN & BRIN).
- Saves query time but may cost extra **storage & maintenance**.

Download Complete SQL & PostgreSQL Notes + Practice Files

You can access the full set of **SQL/PostgreSQL notes** along with practice datasets and queries from this GitHub repository:

👉 [SQL-resources-and-tutorials by akshay-dhage](#)

akshay dhage